

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

CLI To-Do List Manager - Technical Specification & Architecture Document	3
1. Executive Summary & Architecture Overview	3
1.1 Executive Brief	3
1.2 Maturity Assessment	3
1.3 Technical Stack	3
1.4 Architectural Constraints	3
1.5 Critical Dependencies	3
2. Architecture Workflows	4
2.1 Data Model: Task Management	6
2.2 Requirements Traceability Matrix	7
2.3 Task Operation Workflow	7
2.4 CLI Interaction Sequence	8
3. Detailed Technical Specifications & Business Rules	9
3.1 Requirements Traceability	9
3.2 Security Rules	11
3.3 Data Models	11
4. Project Governance & Structural Gaps	11
4.1 Structural Gaps	11
4.2 Remediation & Workflow	11
5. Technical & Domain Glossary (Terminology Reference)	12

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

A Python-based CLI To-Do List Manager designed for local task orchestration. The system implements a flat-file data pattern using a JSON store at `~/ . todos . json` to manage task lifecycles (creation, completion, and deletion) with support for both human-readable and machine-readable output formats.

1.2 Maturity Assessment

The specification is structurally complete in terms of functional mapping but requires REFINEMENT. While the core logic is well-defined, there are unresolved technical uncertainties regarding error handling for corrupted JSON files, invalid ID inputs, and concurrent file access. The absence of a 'Scope & Out-of-Scope' definition creates a risk of scope creep regarding cloud synchronization or categorization features.

1.3 Technical Stack

- **Language:** Python 3.8+
- **Storage:** Local JSON file (`~/ . todos . json`)
- **Interface:** Command Line Interface (CLI)

1.4 Architectural Constraints

- **Execution Latency:** All basic operations (add, list, complete, remove) must execute in under 2 seconds.
- **Scalability Threshold:** Zero performance degradation up to 1000+ tasks in the JSON store.
- **Data Storage:** Mandatory local persistence at `~/ . todos . json`.
- **ID Generation:** Strict unique sequential integer assignment.
- **Output Format:** Dual-mode output (Standard text and Valid JSON via `--json` flag).
- **User Access:** Limited to single-user local environment without network connectivity.

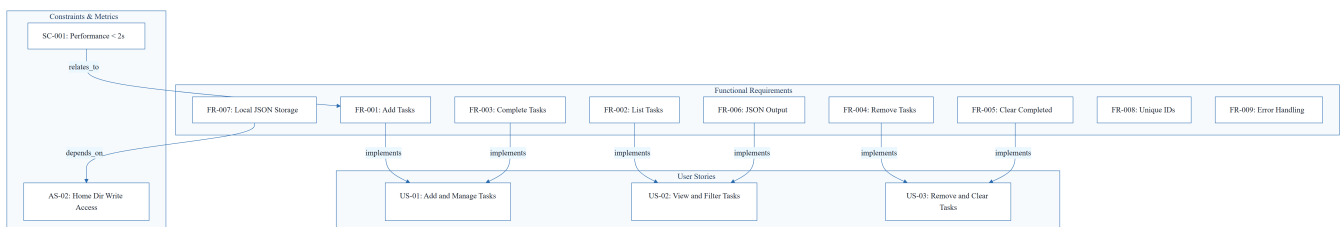
1.5 Critical Dependencies

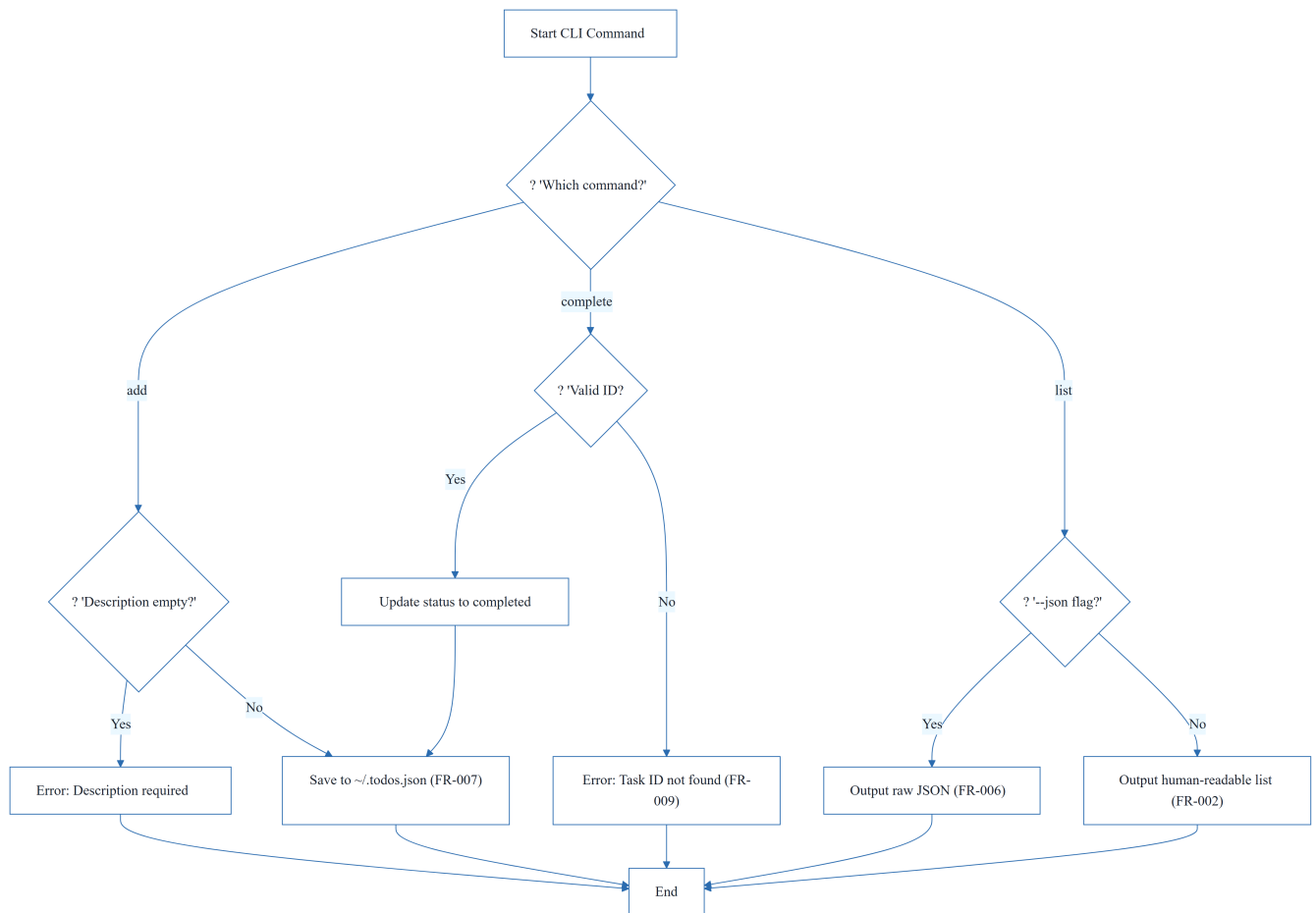
- Write access to the system home directory for `~/ . todos . json` creation.
- Python 3.8+ runtime environment.
- Strict referential integrity between the CLI commands and the ENT-TASK entity attributes (`id`, `description`, `completed`, `created_at`).

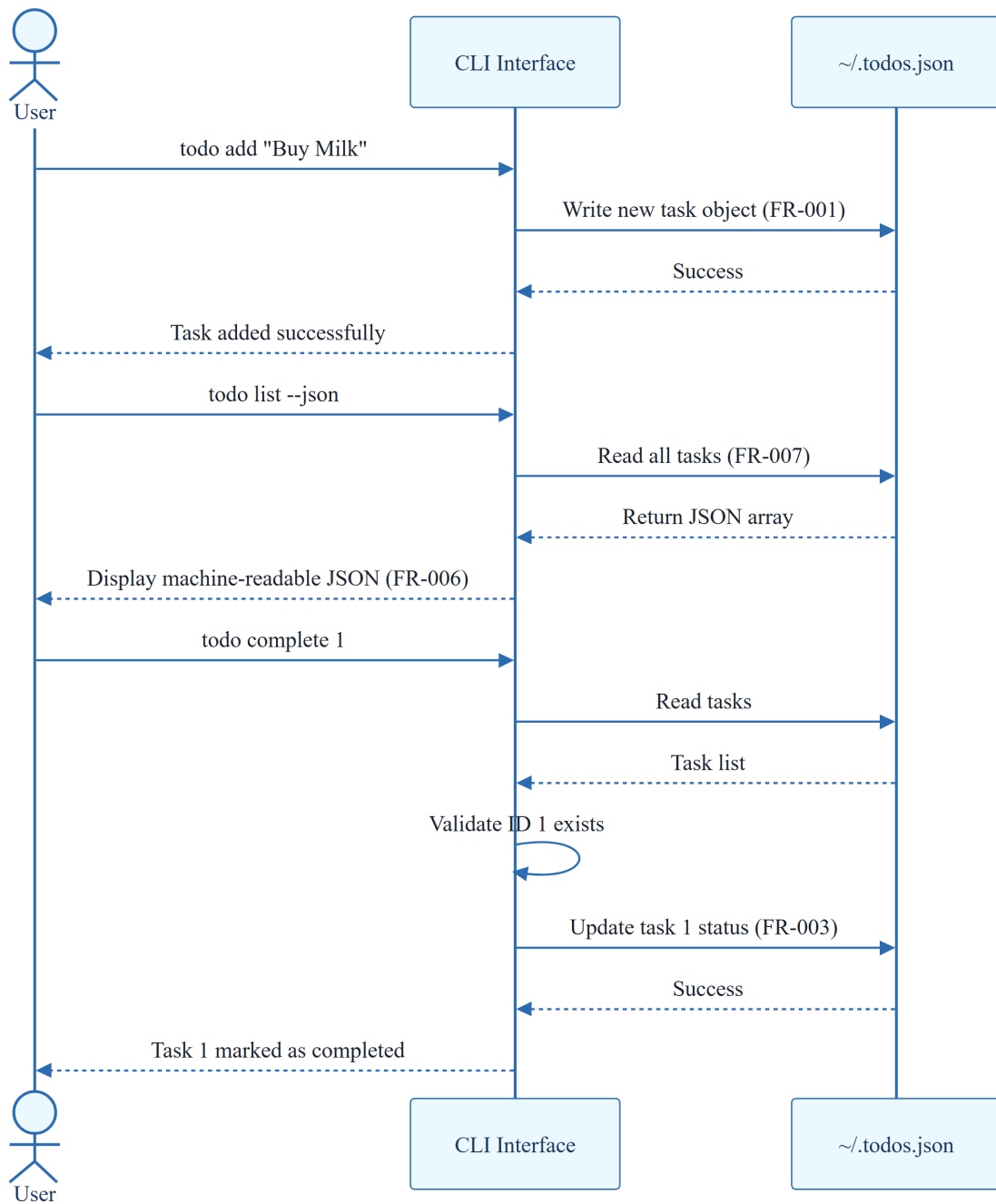
- JSON standard compliance for machine-readable output parseability.

2. Architecture Workflows

ENT-TAS]		
int	id	PK
string	description	
boolean	completed	
timestamp	created_at	





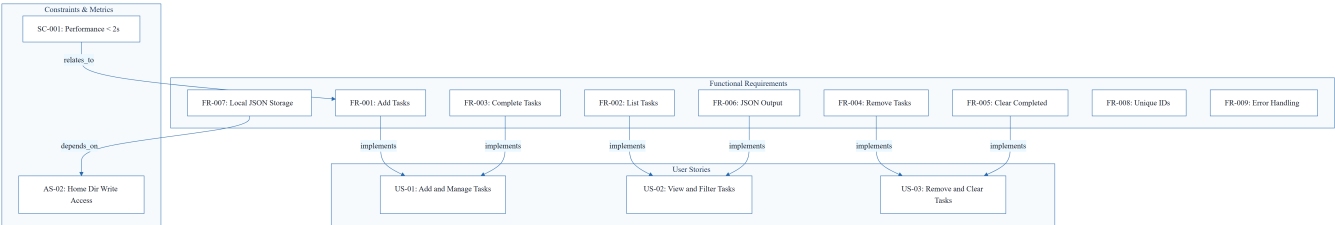


& Visual Diagrams

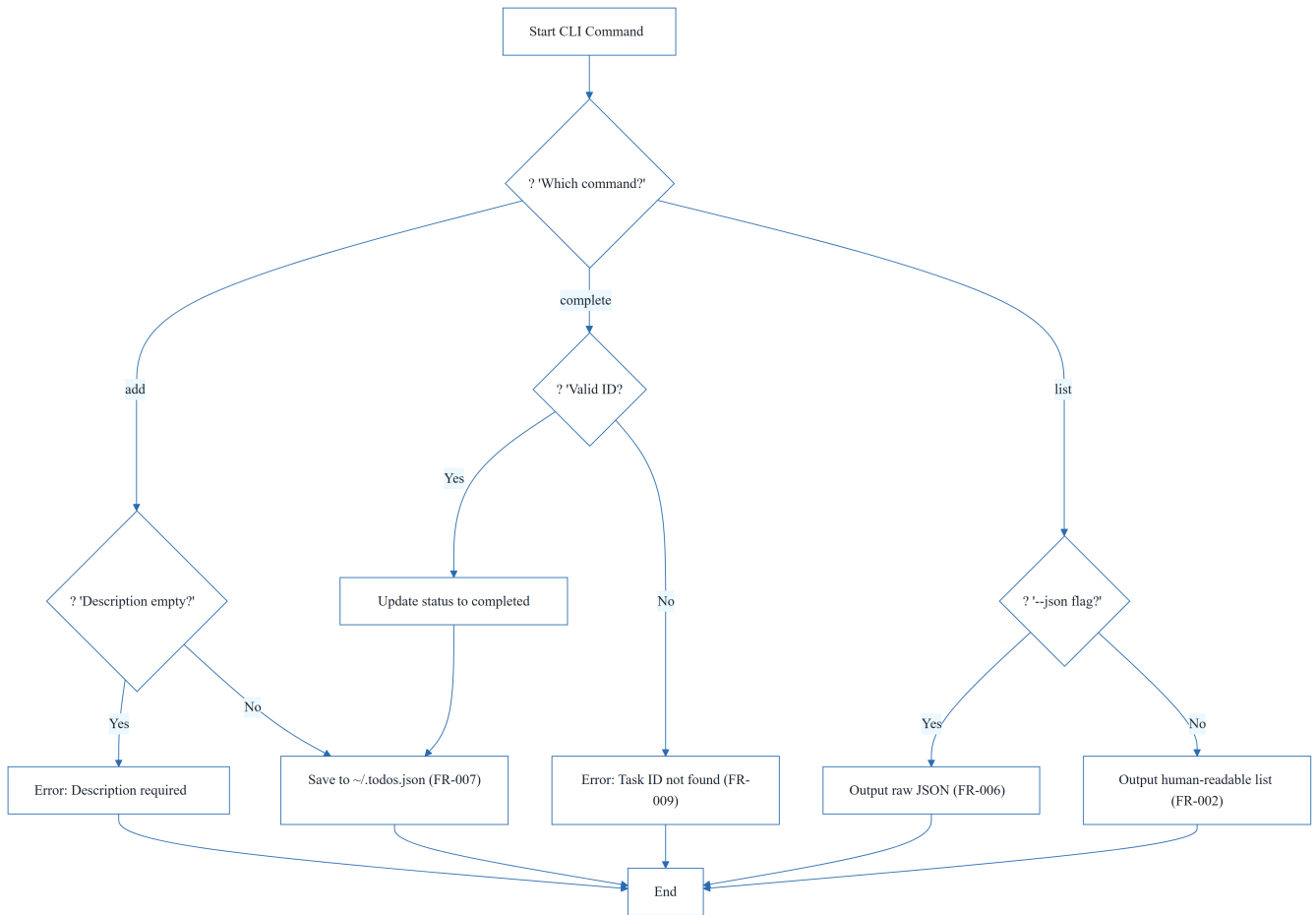
2.1 Data Model: Task Management

ENT-TAS]		
int	id	PK
string	description	
boolean	completed	
timestamp	created_at	

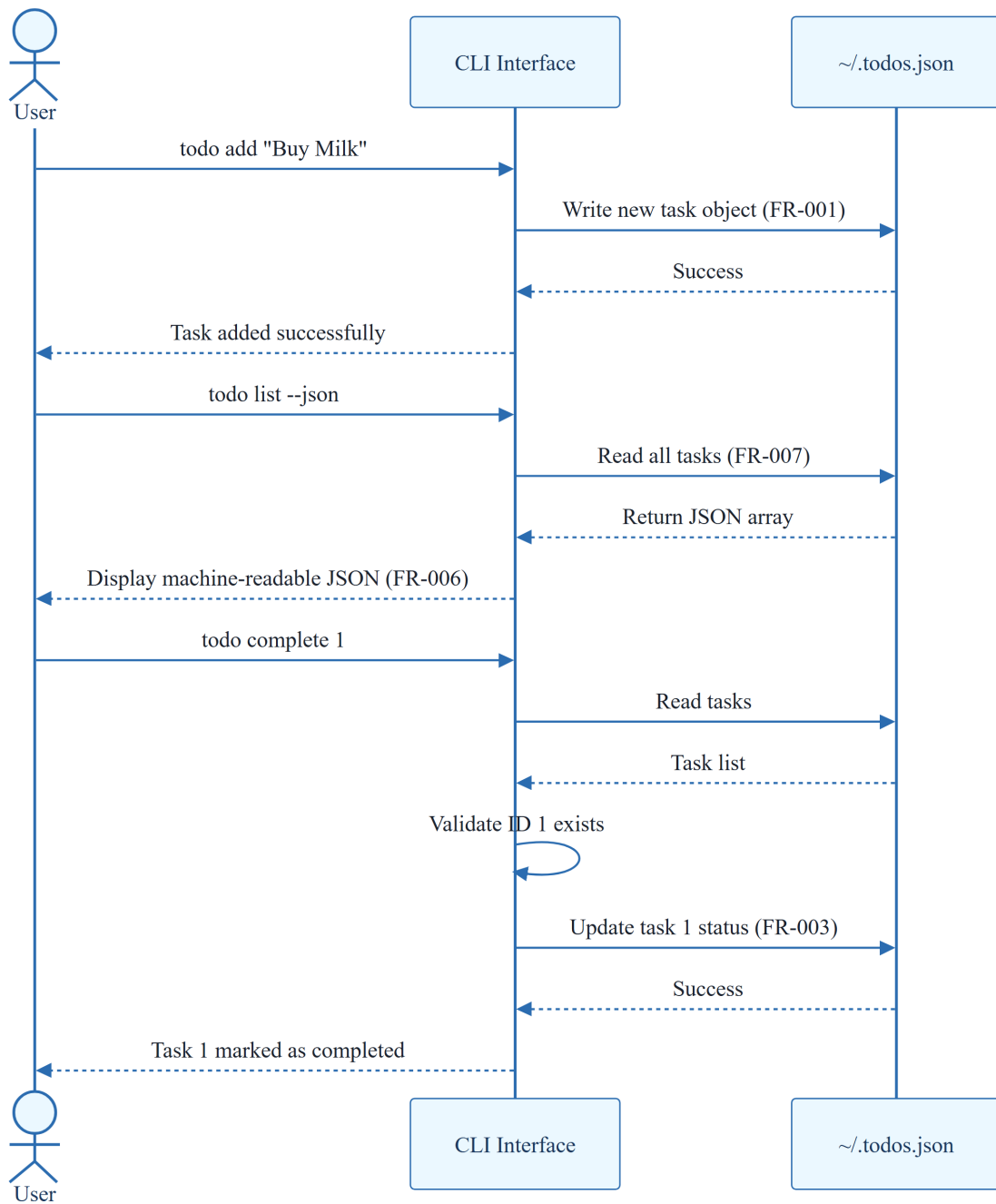
2.2 Requirements Traceability Matrix



2.3 Task Operation Workflow



2.4 CLI Interaction Sequence



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Type	Description	Source / Relation
US-01	User Story	Quickly capture tasks and manage completion status via CLI.	P1

ID	Type	Description	Source / Relation
US-02	User Story	View tasks in readable or machine-readable JSON format.	P2
US-03	User Story	Remove individual tasks or clear all completed tasks.	P3
FR-001	Functional	System MUST allow users to add new tasks via <code>add</code> command.	Implements US-01
FR-002	Functional	System MUST display all tasks (ID, desc, status) via <code>list</code> command.	Implements US-02
FR-003	Functional	System MUST allow users to mark tasks as completed via <code>complete</code> command.	Implements US-01
FR-004	Functional	System MUST allow users to remove individual tasks via <code>remove</code> command.	Implements US-03
FR-005	Functional	System MUST allow users to clear all completed tasks via <code>clear</code> command.	Implements US-03
FR-006	Functional	System MUST support JSON output format via <code>--json</code> flag.	Implements US-02
FR-007	Functional	System MUST store all task data in a local JSON file at <code>~/todos.json</code> .	Depends on AS-02
FR-008	Functional	System MUST assign unique sequential IDs to tasks.	-
FR-009	Functional	System MUST handle errors gracefully with user-friendly messages.	-
ENT-TASK	Entity	Task: id (int), description (string), completed (bool), created_at (timestamp).	-
SC-001	Success Criterion	Basic operations must execute in under 2 seconds.	Relates to FR-001
SC-002	Success Criterion	Handle 1000+ tasks without performance degradation.	-
SC-003	Success Criterion	95% of users perform basic operations successfully on first attempt.	-

ID	Type	Description	Source / Relation
SC-004	Success Criterion	JSON output is valid and parseable by standard parsers.	-
SC-005	Success Criterion	Error messages reduce user confusion by 80%.	-
AS-01	Assumption	Users have Python 3.8+ installed on their system.	-
AS-02	Assumption	Users have write access to their home directory.	-

3.2 Security Rules

- **Local Access Only:** The system is designed for a single-user local environment. No network connectivity or remote access is implemented.
- **File Permissions:** Reliance on OS-level home directory permissions to protect `~/todos.json`.

3.3 Data Models

Entity: Task (ENT-TASK)

- `id`: Unique integer identifier (Auto-incremented).
- `description`: String describing the task.
- `completed`: Boolean indicating completion status (Default: `false`).
- `created_at`: Timestamp of task creation.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap ID	Missing Section	Priority	Remediation Advice
GAP-01	Scope & Out-of-Scope	MEDIUM	Explicitly define what the CLI tool will NOT do (e.g., no cloud sync, no categories).

4.2 Remediation & Workflow

To address the identified gaps and open questions, the following workflow is proposed:

1. **Scope Definition:** Define a "Non-Goals" list to prevent scope creep.
2. **Error Handling Specification:** Define specific behaviors for:

- Invalid IDs (FR-009).
- Corrupted JSON files.
- Empty task descriptions.

3. **Concurrency Strategy:** Determine if file locking is required for concurrent access to `~/todos.json`.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
ID	BUSINESS_DOMAIN	FR-008	A unique sequential integer assigned to each task for individual reference and targeting.
JSON	TECHNICAL_STACK	FR-007	The structured data format used for local persistence in the home directory and as an optional machine-readable output.
Python 3.8	TECHNICAL_STACK	AS-01	The minimum required runtime environment installed on the host system to execute the application.
todo clear	BUSINESS_DOMAIN	FR-005	An operation that purges all items marked as finished from the persistence layer while preserving those with a pending status.
todo list	BUSINESS_DOMAIN	FR-002	A command that retrieves and displays all stored entries including their unique identifiers, descriptions, and completion states.